

Student 13 **Jayaraman Renuka Devi** 192573007RD

5 / 9 submitted

Q1. Banking Transaction System – Exception Handling

CODE

```
import java.util.Scanner;
import java.util.InputMismatchException;
public class Main{
    static class InsufficientBalanceException extends Exception{
        public InsufficientBalanceException(String message){
            super(message);
        }
    }
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        try{
            double balance=sc.nextDouble();
            double withdrawl=sc.nextDouble();
            if(withdrawl<=0){
                System.out.println("Invalid withdrawl amount");
            }
            else if(withdrawl>balance){
                throw new InsufficientBalanceException("Insufficient Balance");
            }
            else{
                balance=balance-withdrawl;
                System.out.println("Withdrawal Successful");
                System.out.println("Remaining Balance: " + balance);
            }
        }
        catch(InsufficientBalanceException e){
            System.out.println(e.getMessage());
        }
        catch(InputMismatchException e){
            System.out.println("Invalid numeric input");
        }
        finally{
            System.out.println("Transaction Completed");
        }
    }
}
```

INPUT

10000
12000

OUTPUT

Insufficient Balance
Transaction Completed

Q2. Hospital Patient Registration – User-Defined Exception

CODE

```
import java.util.Scanner;
import java.util.InputMismatchException;
public class Main{
    static class InvalidAgeException extends Exception{
        public InvalidAgeException(String message){
            super(message);
        }
    }
    static class InvalidTemperatureException extends Exception{
        public InvalidTemperatureException(String message){
            super(message);
        }
    }
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        try{
            String name=sc.next();
            int age=sc.nextInt();
            double temperature=sc.nextDouble();
            if(age<18){
                throw new InvalidAgeException("InvalidAgeException");
            }
            if(temperature<95 || temperature>104){
                throw new InvalidTemperatureException("InvalidTemperatureException");
            }
            System.out.println("Patient registered successfully");
        }
        catch(InvalidAgeException e){
            System.out.println(e.getMessage());
        }
        catch(InvalidTemperatureException e){
            System.out.println(e.getMessage());
        }
        catch(InputMismatchException e){
            System.out.println("Invalid Input");
        }
        finally{
            System.out.println("Registration Completed");
        }
    }
}
```

INPUT

ravi
45
105.2

OUTPUT

InvalidTemperatureException
Registration Completed

Q3. E-Commerce Order Processing – Built-in Exceptions

CODE

```
import java.util.Scanner;
import java.util.InputMismatchException;
public class main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        String[] products={"Laptop","Phone","Headphones","Keyboard"};
        double[] prices={50000,20000,3000,1500};
        try{
            int index=sc.nextInt();
            int quantity=sc.nextInt();
            try{
                if(quantity<=0){
                    System.out.println("Invalid quantity");
                }
                else{
                    double total=prices[index]*quantity;
                    System.out.println("Total Cost:: " + total);
                }
            }catch(ArrayIndexOutOfBoundsException e){
                System.out.println("ArrayIndexOutOfBoundsException handled");
            }
        }
        catch(InputMismatchException e){
            System.out.println("InputMismatchException handled");
        }
        System.out.println("Transaction Completed");
    }
}
```

INPUT

5
2

OUTPUT

ArrayIndexOutOfBoundsException handled
Transaction Completed

Q4. Railway Ticket Booking – Multithreading and Synchronization

— Not Submitted —

Q5. Bank Account – Concurrent Withdrawals

— Not Submitted —

Q6. Online Food Delivery – Multiple Threads and Execution Order

CODE

```
public class main{
    static class OrderProcessing extends Thread{
        public void run(){
            System.out.println("Order Processing Completed");
        }
    }
    static class PaymentProcessing extends Thread{
        public void run(){
            System.out.println("Payment Processing Completed");
        }
    }
    static class DeliveryAssignment extends Thread{
        public void run(){
            System.out.println("Delivery Assignment Completed");
        }
    }
    public static void main(String[] args) throws InterruptedException{
        System.out.println("Independent Execution: ");
        Thread order1=new OrderProcessing();
        Thread payment1=new PaymentProcessing();
        Thread delivery1=new DeliveryAssignment();
        order1.start();
        payment1.start();
        delivery1.start();
        order1.join();
        payment1.join();
        delivery1.join();
        System.out.println("\nControlled Execution: ");
        Thread order2=new OrderProcessing();
        Thread payment2=new PaymentProcessing();
        Thread delivery2=new DeliveryAssignment();
        order2.start();
        order2.join();
        payment2.start();
        payment2.join();
        boolean paymentSuccess=true;
        if(paymentSuccess){
            delivery2.start();
            delivery2.join();
        }
        else{
            System.out.println("Payment Failed-Delivery Assignment Cancelled");
        }
        System.out.println("Food Delivery Process Completed");
    }
}
```

OUTPUT

```
Independent Execution:
Order Processing Completed
Delivery Assignment Completed
Payment Processing Completed

Controlled Execution:
Order Processing Completed
Payment Processing Completed
Delivery Assignment Completed
Food Delivery Process Completed
```

Q7. Hospital Emergency System – Thread Priority

CODE

```
import java.util.Scanner;
public class main{
    static class EmergencyPatient extends Thread{
        public EmergencyPatient(){
            setName("Emergency");
            setPriority(Thread.MAX_PRIORITY);
        }
        public void run(){
            System.out.println("Emergency Patient Processing");
        }
    }
    static class GeneralPatient extends Thread{
        public GeneralPatient(){
            setName("General");
            setPriority(Thread.NORM_PRIORITY);
        }
        public void run(){
            System.out.println("General Patient processing");
        }
    }
    static class Billing extends Thread{
        public Billing(){
            setName("Billing");
            setPriority(Thread.MIN_PRIORITY);
        }
        public void run(){
            System.out.println("Billing Processing");
        }
    }
    public static void main(String[] args) throws InterruptedException{
        EmergencyPatient emergency=new EmergencyPatient();
        GeneralPatient general= new GeneralPatient();
        Billing billing=new Billing();
        System.out.println("Thread Priority:");
        System.out.println("Emergency: " + emergency.getPriority());
        System.out.println("General: " + general.getPriority());
        System.out.println("Billing: " + billing.getPriority());
        emergency.start();
        general.start();
        billing.start();
        emergency.join();
        general.join();
        billing.join();
        System.out.println("All processing completed");
        System.out.println("\nStrict Execution Sequence: ");
        EmergencyPatient e2=new EmergencyPatient();
        GeneralPatient g2=new GeneralPatient();
        Billing b2=new Billing();
        e2.start();
        e2.join();
        g2.start();
        g2.join();
        b2.start();
        b2.join();
        System.out.println("Strict sequence completed");
    }
}
```

OUTPUT

```
Thread Priority:
Emergency: 10
General: 5
Billing: 1
```

```
Emergency Patient Processing
Billing Processing
General Patient processing
All processing completed
```

```
Strict Execution Sequence:
Emergency Patient Processing
General Patient processing
Billing Processing
Strict sequence completed
```

Q8. Manufacturing Plant – Producer-Consumer Problem

— Not Submitted —

Q9. Online Banking – Exception Handling with Multiple Threads

— Not Submitted —